Feature Description

Ethereum Token Spam Detection

FinTech-540 Project

Project Overview

This project builds a supervised binary classifier to distinguish spam tokens (label = 0) from legitimate tokens (label = 1) on the Ethereum blockchain. Data covers **1,000 consecutive blocks** ($\approx$ 20 million token transfer events) starting at block 20,000,000.

Labels are defined as follows:

- **Legit (1):** Contract address appears in the verified token list (`token_labels.csv`).
- **Spam (0):** Token shares the same symbol as a verified token but uses a different contract address (symbol collision).
- **Unlabeled (NaN):** Unverified tokens without a known collision — excluded from supervised training.

All features are aggregated at the **token (contract address) level** from the raw transfer-event data. The final labeled dataset contains **3,606 tokens** (spam = 2,007 | legit = 1,599).

Raw Transfer Data Columns

The primary data file is `transfers_20000000.parquet`. Below are the raw columns used as inputs for feature engineering.

Column	Description
`blockNum`	Block number (hexadecimal string) in which the transfer was included
`hash`	Transaction hash uniquely identifying the on-chain transaction
`from`	Sender Ethereum address (20-byte hex)
`to`	Recipient Ethereum address (20-byte hex)
`value`	Transfer amount in decimal token units (float; null for ERC-721/1155)
`asset`	Token symbol as reported by the Alchemy API (e.g., USDT, PEPE)
`category`	Transfer category: `erc20`, `token`, `erc721`, `erc1155`, `specialnft`
`rawContract.address`	Contract address of the token (lowercase hex)
`rawContract.decimal`	Token decimal precision (e.g., 18 for most ERC-20 tokens)
`rawContract.value`	Raw transfer amount as a hex string before decimal conversion
`metadata.blockTimestamp`	ISO timestamp of the block
`tokenId`	NFT token ID (only for ERC-721 / ERC-1155; null for ERC-20)

Note: only transfers with category in {erc20, token, erc721, erc1155, specialnft} are used (≈ 2.16 million rows after filtering).

Engineered Features

All 27 features below are computed per unique `contract_addr` (i.e., one row per token) and used as the model input matrix X.

Feature	Calculation	Spam signal / interpretation
Activity & volume		
`n_transfers`	`COUNT(hash)` per token	Spam tokens typically have a low total number of transfers (one-off airdrop)
`n_unique_senders`	`NUNIQUE(from)` per token	Spam is sent by very few addresses (often just 1 bot); highest correlation with label (r=0.61)
`n_unique_receivers`	`NUNIQUE(to)` per token	Spam airdrops hit many receivers; combined with senders gives the airdrop ratio
Transfer value		
`value_mean`	`MEAN(value)` per token	Less discriminative alone; useful in combination with std
`value_std`	`STD(value)` per token	Spam uses uniform fixed amounts (std ≈ 0); legit tokens have varied transfer sizes
`value_null_ratio`	`MEAN(value IS NULL)` per token	Spam tokens (especially fake ERC-20s) often lack decimal metadata, producing null values
`zero_value_ratio`	`MEAN(value == 0)` per token	Spam airdrops frequently send zero-value transfers to claim wallet attention
`unique_values_count`	`NUNIQUE(value)` per token	Spam uses very few distinct amounts (often exactly 1); legit tokens show diverse amounts
Block distribution		
`n_distinct_blocks`	`NUNIQUE(block_int)` per token	Spam is deployed in a burst of 1–2 blocks; ≈78% of spam in a single block vs 12% for legit
`block_range`	`MAX(block_int)` − `MIN(block_int)`	Complementary to `n_distinct_blocks`; 0 means all transfers happened in one block
Ratio & derived		
`sender_receiver_ratio`	*senders / receivers*	Core airdrop signal: few senders → many receivers gives ratio $\ll 1$; spam median ≈ 0.001
`transfers_per_block`	*n_transfers / n_blocks*	High value indicates a burst (many transfers in one block), characteristic of automated airdrops
Concentration		
`top1_sender_share`	max(per-sender count) / *n_transfers*	Close to 1.0 if a single bot drove all transfers; strong indicator of automated spam
`receiver_concentration`	Gini coeff. of transfers-per-receiver	High Gini ($\to 1$) means transfers are concentrated on few receivers; low means evenly spread
External / categorical		
`category_entropy`	Shannon entropy of the transfer category distribution	Computed as $-\sum p_i \log p_i$ over category proportions; low entropy = homogeneous transfers
`sender_is_labeled`	Binary: 1 if any sender address appears in	Legit tokens are more likely sent by known DEX routers or

Table 1: Description of engineered features used in the spam detection model

Label Construction & Leakage Note

Two intermediate columns are computed during preprocessing but **excluded from the feature matrix** because they directly encode the label:

Column	Why excluded
`symbol_collision`	Binary flag set to 1 when a token's symbol matches a verified token but the contract address differs. This is the *definition* of the spam label (label = 0), so including it would be direct data leakage.
`is_verified`	Binary flag set to 1 when the contract address appears in `token_labels.csv`. This is the *definition* of the legit label (label = 1).

Preprocessing Steps

Before model training, the following transformations are applied (see `preprocessing.ipynb`):

1. **Imputation:** Missing values filled with the column median (no nulls were found in the labeled set).
2. **Log transform:** `log1p` applied to 18 right-skewed features: `n_transfers`, `n_unique_senders`, `n_unique_receivers`, `value_mean`, `value_std`, `sender_receiver_ratio`, `transfers_per_block`, `n_distinct_blocks`, `block_range`, `unique_values_count`, `in_degree_mean`, `out_degree_max`, `n_connected_components`, `inter_block_time_var`, `burst_duration_ratio`, `mev_sandwich_count`, `mev_frontrun_count`, `gas_price_at_deploy`.
3. **Scaling:** `RobustScaler` (centers on median, scales by IQR) fit on the training set only and applied to validation and test sets — prevents data leakage through the scaler.
4. **Split:** Stratified 70 / 15 / 15 train / val / test split. Train = 2,524 | Val = 541 | Test = 541.

Note: tree-based models (Random Forest, XGBoost, LightGBM) are trained on the ***unscaled*** *log-transformed splits since they are scale-invariant. Linear models (Logistic Regression, Naive Bayes) use the scaled splits.*